Reviewer Pack — Real Rank of Depth-3 AC⁰ Circuits for PARITY is Ω(log n)

Entry be45483362ed · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 16:11:45 UTC

Real Rank of Depth-3 AC⁰ Circuits for PARITY is Ω(log n)

Entry ID: `be45483362ed` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-11 16:11:45 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): AC⁰ Circuits Computing PARITY

Statement:

> For any depth-3 AC⁰ circuit C computing PARITY on n bits, the real rank of the matrix M_C whose entries are the coefficients of the polynomial representing C is at least Ω(log n).

Rationale (proposer's reasoning):

> Real rank captures geometric complexity of polynomial representations, while AC⁰ circuits for PARITY are inherently linear but constrained by depth. Linking them exposes trade-offs between algebraic structure and circuit depth.

Taxonomy category: `AC0_PARITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c38fb9c8874ea133`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.3s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_ac0_circuit(n):
    circuit = []
    for _ in range(2**n):
        gate_type = random.choice(['AND', 'OR'])
        inputs = [random.randint(0, 1) for _ in range(n)]
        if gate_type == 'AND':
            output = all(inputs)
        else:
            output = any(inputs)
        circuit.append((gate_type, inputs, output))
    return circuit

def polynomial_representation(circuit):
    n = len(circuit[0][1])
    poly = [Fraction(0)] * (2**n)
    for gate in reversed(circuit):
        gate_type, inputs, output = gate
        if gate_type == 'AND':
            term = Fraction(1)
            for bit in inputs:
                term *= Fraction(bit)
            poly[output] += term
        else:
            term = Fraction(1)
            for bit in inputs:
                term *= 1 - Fraction(bit)
            poly[output] -= term
    return poly

def matrix_from_polynomial(poly, n):
    m = len(poly)
    matrix = [[Fraction(0)] * (m + 1) for _ in range(m)]
    for i in range(m):
        for j in range(n):
            if i & (1 << j):
                matrix[i][j] += poly[i]
        matrix[i][-1] -= poly[i]
    return matrix

def qr_decomposition(matrix):
    m, n = len(matrix), len(matrix[0])
```

```
    Q = [[Fraction(0)] * n for _ in range(m)]
    R = [[Fraction(0)] * n for _ in range(n)]

    for k in range(n):
        norm = Fraction(0)
        for i in range(k, m):
            norm += matrix[i][k] ** 2
        norm = norm ** Fraction(1, 2)

        if norm == Fraction(0):
            raise ValueError("Matrix is singular")

        Q[k][k] = matrix[k][k] / norm
        R[0][k] = matrix[k][k]

        for j in range(k + 1, n):
            R[k][j] = Fraction(0)
            for i in range(k, m):
                R[k][j] += Q[i][k] * matrix[i][j]
            for i in range(k, m):
                matrix[i][j] -= Q[i][k] * R[k][j]

        for i in range(k + 1, m):
            Q[i][k] = -matrix[i][k] / norm
            for j in range(k + 1, n):
                Q[i][j] += Q[i][k] * R[k][j]

    return Q, R

def real_rank(matrix):
    _, R = qr_decomposition(matrix)
    rank = sum(1 for row in R if any(x != Fraction(0) for x in row))
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    circuit = generate_ac0_circuit(n)
    poly = polynomial_representation(circuit)
    matrix = matrix_from_polynomial(poly, n)
    rank = real_rank(matrix)

    metric_name = "real_rank"
    metric_value = rank
    instances_tested = 1
    conjecture_holds = rank >= 0.3 * math.log2(n)
    counterexample = "" if conjecture_holds else f"rank={rank}, expected ≥{0.3 * math.log2(n)}"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }
```

```python
if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = (sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results)) *
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture
        print(f"RESULT: FALSIFIED counterexample=\"rank too small\" first_failing_seed={first_fail
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test timed out without producing results, preventing evaluation of support fraction or counterexamples. | next: Run test with extended timeout and higher resource allocation

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	108224
2	preregistration	ollama_remote	qwen3:8b	0	0	23889
3	novelty	ollama_remote	qwen3:8b	0	0	20424
4	novelty	ollama_remote	qwen3:8b	0	0	13207
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14609

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12969
7	judge	ollama_remote	qwen3:8b	0	0	58312

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 251634 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/be45483362ed.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/be45483362ed.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/be45483362ed.tar.gz` (if generated)